UNIVERSITÄT TRIER

Übungsblatt 1

Nebenläufigkeit mit POSIX Threads

Umfang: 10 Punkte **Abgabe:** 06.12.2026, 24:00 Uhr (MEZ)
Bearbeitung: ausschließlich einzeln; keine Gruppenarbeit

Ziel und Rahmenbedingungen

Sie untersuchen Data Races, blockierende Synchronisation, Deadlocks und phasenweise parallele Berechnungen. Begründen Sie die Korrektheit Ihrer Lösungen unabhängig von einer bestimmten Ausführungsreihenfolge.

Verwenden Sie C oder C++ und ausschließlich POSIX Threads für die Nebenläufigkeit: `pthread_create`, `pthread_join`, `pthread_mutex_*` und `pthread_cond_*`. Implementieren Sie Puffer und Barriere selbst. C++-Nebenläufigkeitsbibliotheken, OpenMP, fertige nebenläufige Container und Bibliotheksbarrieren sind nicht zugelassen.

Prüfen Sie Rückgabewerte. Beenden Sie alle Threads geordnet und geben Sie Ressourcen frei. Aktives Warten und Synchronisation durch Wartezeiten sind nicht erlaubt. Kurze Verzögerungen dürfen ausschließlich Arbeit oder unterschiedliche Ablaufgeschwindigkeiten simulieren.

Aufgabe 1 · Gemeinsamer Zähler — 2 Punkte

Mehrere Threads erhöhen einen Zähler jeweils N-mal. Threadzahl und N sind konfigurierbar. Implementieren Sie drei Varianten:

1. Einen absichtlich ungeschützten Zugriff als Fehlerdemonstration.
2. Eine Mutex-geschützte Erhöhung bei jedem Zugriff.
3. Lokale Zähler pro Thread mit anschließender sicherer Zusammenführung.

Prüfen Sie das Ergebnis gegen Threadzahl $\times$ N. Vergleichen Sie die beiden korrekten Varianten bei verschiedenen Threadzahlen. Erklären Sie Data Races, undefiniertes Verhalten und den Einfluss konkurrierender Zugriffe. Auch ein korrekt erscheinendes Ergebnis macht die erste Variante nicht korrekt; ihre Laufzeit ist keine belastbare Vergleichsbasis.

Aufgabe 2 · Beschränkter Puffer — 2,5 Punkte

Implementieren Sie einen FIFO-Puffer mit konfigurierbarer Kapazität für mehrere Produzenten und Konsumenten. Nutzen Sie einen Mutex und Bedingungsvariablen.

Bei vollem Puffer warten Produzenten, bei leerem Puffer warten Konsumenten. Nachdem alle Produzenten beendet sind, wird der Puffer geschlossen. Verbleibende Elemente werden noch entnommen; anschließend müssen alle Konsumenten terminieren. Legen Sie fest, wie sich ein Einfügeversuch nach dem Schließen verhält.

Vergeben Sie eindeutige Elementkennungen und prüfen Sie, dass jedes erzeugte Element genau einmal entnommen wurde. Testen Sie Kapazität eins sowie unterschiedliche Kombinationen von Produzenten und Konsumenten. Erläutern Sie Wartepräditate, erneute Prüfung nach dem Aufwachen und das Aufwecken beim Schließen.

Aufgabe 3 · Kontotransfers

2,5 Punkte

Mehrere Threads führen Überweisungen zwischen zufällig gewählten, verschiedenen Konten aus. Verwenden Sie ganzzahlige Guthaben und einen eigenen Mutex pro Konto.

Eine Überweisung ist nur bei ausreichender Deckung zulässig. Prüfung und beide Kontobewegungen müssen eine geschützte Operation bilden. Die Summe aller Guthaben bleibt erhalten; kein Guthaben darf negativ werden. Wählen und begründen Sie eine Strategie zur deadlockfreien Sperrung beider Konten.

Prüfen Sie die Invarianten nach dem Ende aller Threads. Zeigen Sie einen möglichen Deadlock bei ungeeigneter Sperrreihenfolge und erläutern Sie, wie während laufender Transfers eine konsistente Gesamtübersicht möglich wäre. Verwenden Sie pro Thread einen eigenen Zufallszahlengenerator mit einem festen, aus der Threadnummer abgeleiteten Seed. Jeder Thread erzeugt eine feste Anzahl von Überweisungsvorschlägen; seine Zufallsentscheidungen dürfen nicht vom Erfolg vorheriger Überweisungen abhängen. Dadurch lassen sich die vorgeschlagenen Überweisungen pro Thread reproduzieren. Die tatsächliche Ausführungsreihenfolge und die Anzahl erfolgreicher Überweisungen können zwischen Programmläufen variieren.

Aufgabe 4 · Wiederverwendbare Barriere

3 Punkte

Teil A: Synchronisationsmechanismus. Implementieren Sie mit Mutex und Bedingungsvariable eine Barriere für eine feste positive Teilnehmerzahl. Kein Thread darf eine Runde verlassen, bevor alle Teilnehmer angekommen sind. Dieselbe Barriere muss über viele Runden wiederverwendbar sein. Nutzen Sie weder `pthread_barrier_t` noch andere fertige Barrieren.

Begründen Sie insbesondere, wie aufeinanderfolgende Runden sicher getrennt werden, wenn schnelle Threads bereits erneut eintreffen. Testen Sie viele kurze Runden, ungleich schnelle Teilnehmer und einen einzelnen Teilnehmer. Alle Teilnehmer rufen die Barriere pro Runde genau einmal auf; ein vorzeitiger Ausstieg ist nicht zu unterstützen.

Teil B: Parallele Gitterberechnung. Berechnen Sie auf zwei zweidimensionalen Puffern wiederholt für jede innere Zelle den Mittelwert ihrer vier direkten Nachbarn:

$$u_{r,c}^{(k+1)} = \frac{u_{r-1,c}^{(k)} + u_{r+1,c}^{(k)} + u_{r,c-1}^{(k)} + u_{r,c+1}^{(k)}}{4}$$

Die Randwerte bleiben konstant. Initialisieren Sie beide Puffer mit identischen Randwerten. Verteilen Sie innere Zeilen auf die Threads und führen Sie eine feste konfigurierbare Anzahl von Iterationen aus:

1. Jeder Thread berechnet seinen Bereich im Zielpuffer.
2. Alle Threads erreichen eine erste Barriere.
3. Ein festgelegter Thread vertauscht die Rollen der beiden Puffer.
4. Eine zweite Barriere gibt den Beginn der nächsten Iteration frei.

Auch Threads ohne zugeteilte Zellen nehmen an beiden Barrieren teil. Vergleichen Sie das Ergebnis mit einer sequenziellen Referenz innerhalb einer angegebenen numerischen Toleranz. Messen Sie den Speedup gegenüber der sequenziellen Ausführung für verschiedene Gittergrößen und Threadzahlen. Erklären Sie den Einfluss der Synchronisationskosten.

Abgabe und Bewertung

Reichen Sie Quellcode, Bau- und Ausführungsanleitung und einen Bericht von etwa 3–5 Seiten bis zum **06.12.2026, 24:00 Uhr (MEZ)** ein. Bearbeiten Sie alle Aufgaben einzeln; Gruppenarbeit ist nicht zulässig. Präsentieren Sie Ihre Lösungen nach der Abgabe in einer Besprechung in **etwa 5–10 Minuten**. Der Termin wird gesondert bekannt gegeben.

Erläutern Sie pro Aufgabe gemeinsame Daten, Invarianten und Synchronisationsentscheidungen. Bewertet werden Korrektheit, Begründung und Überprüfung innerhalb der jeweiligen Aufgabenpunkte.

Dokumentieren Sie Hardware, Betriebssystem, Compiler, Optimierung und Messparameter. Berichten Sie Median und Streuung wiederholter Messungen ohne Konsolenausgaben. Nutzen Sie, soweit verfügbar, **ThreadSanitizer (TSan)**, **Valgrind Helgrind** oder **Valgrind DRD**. Messen Sie Laufzeiten separat ohne Race Detector. Ein unauffälliger Testlauf ist kein Korrektheitsbeweis.